



Unión Europea

Fondo Social Europeo

El FSE invierte en tu futuro

MVC

Vida y Milagros

Pepe

INSTITUT



**GENERALITAT
VALENCIANA**

Conselleria d'Educació,
Investigació, Cultura i Esport



GOBIERNO
DE ESPAÑA

MINISTERIO
DE EDUCACIÓN, CULTURA
Y DEPORTE

Continguts

1	Conexión	3
2	El MODELO Crear, Modificar, eliminar y Consultar (CRUD)	4
2.1	Acceder a los datos (select * from...)	4
2.2	Borrado de datos (Delete)	5
2.3	Actualizar (UPDATE)	5
2.4	Insertar (CREATE)	6
3	El Controlador	6
3.1	Insertar	7
3.2	Borrar	7
3.3	Editar	7
3.4	Select	8
4	La Vista	8
5	Login & Password	10

1 Conexión

Primero vamos a crear un archivo llamado `conexion.php` en este archivo vamos a conectarnos con el sistema gestor de base de datos que en este caso es mysql, para podernos conectar a mysql necesitamos sus características, en este caso:

- Driver: que es el nombre del sistema gestor de base de datos.
- host: en este caso el host es localhost.
- usuario: Mi caso uso el root. Sino tengo una contraseña dejamos vacío el campo.
- dbName: El nombre de nuestra base de datos.

Ahora vamos a crear una instancia de PDO.

```
// MYSQL: Conexión con la base de datos
class Conexion {
    private $pdo;
    private $host;
    private $nombreBD ;
    private $usuario;
    private $password ;

    public function __construct()
    {
        $this->pdo="";
        $this->host = "localhost";
        $this->nombreBD = "videojuego";
        $this->usuario = "root";
        $this->password = "";
    }

    public function conectar(){

        try{

            $this->pdo = new PDO("mysql:host=$this->host;dbname=$this->
                nombreBD;charset=utf8",$this->usuario, $this->password);
            return $this->pdo;
        } catch (PDOException $e) {
            print " <p class=\"aviso\">Error: No puede conectarse con
                la base de datos. {$e->getMessage()}</p>\n";
            exit;
        }
    }
}
```

2 EL MODELO Crear, Modificar, eliminar y Consultar (CRUD)

El modelo es quien define la lógica de negocio. Son las clases y los métodos que se comunican directamente con la base de datos. Creamos nuestra clase en mi caso `Videojuego.php`, donde nosotros vamos a heredar de `Conexion.php`, creamos los atributos y nuestra conexión PDO, ahora creamos un constructor, por lo tanto accederemos a esta con la palabra reservada `parent`, y asignamos estos dos datos a nuestras variables locales para compartirlos con la clase.

```
require_once 'Conexion.php';

class Videojuego extends Conexion
{
    private $pdo;

    public function __construct()
    {
        parent::__construct();
        $this->pdo=parent::conectar();
    }
    // a partir de aquí iran todas los métodos CRUD.
```

Ahora vamos a obtener todos los datos que están registrados en nuestra tabla, para eso llamamos nuestra conexión y usamos la función **prepare** para preparar nuestra sentencias SQL, después la ejecutamos con el método **execute** para poder obtener los datos usaremos `fetchAll` nos devuelve un `array` que contiene todas las filas de la tabla, consultamos nuestra tabla y obtenemos todos los datos. **try catch** para obtener los errores que se regresen.

2.1 Acceder a los datos (select * from...)

Ahora vamos a obtener todos los datos que están registrados en nuestra tabla, para eso llamamos nuestra conexión y usamos la función `prepare` para preparar nuestra sentencias SQL, después la ejecutamos con el método `execute` para poder obtener los datos usaremos `fetchAll` nos devuelve un `array` que contiene todas las filas de la tabla.

```
// Método Read que devuelve un array con todos los registro de la tabla

public function Listar()
{
    try{
        $query = "SELECT * FROM videojuego";
        $registro = $this->pdo->prepare($query);

        $registro->execute();
```

```
        return $registro->fetchAll();
    } catch (PDOException $e)
    {
        die($e->getMessage());
    }
}
```

Ahora obtenemos un dato en específico de la tabla. Después la ejecutamos con el método `execute` para poder obtener los datos usaremos `fetch`, nos devuelve un array con 1 solo elemento.

```
public function getJuego($id)
{
    try{
        $query = "SELECT * FROM videojuego WHERE id = $id";

        $registro = $this->pdo->prepare($query);

        $registro->execute();
        return $registro->fetch();

    } catch (PDOException $e)
    {
        die($e->getMessage());
    }
}
```

2.2 Borrado de datos (Delete)

```
public function Borrar($d)
{
    try{

        $insercion = $this->pdo->prepare("delete from videojuego where
            titulo=:titulo");
        $insercion->bindParam(':titulo', $d);
        return $insercion->execute();

    } catch(PDOException $e)
    {
        die($e->getMessage());
    }
}
```

2.3 Actualizar (UPDATE)

```
public function Editar($i,$t,$g,$p)
{
    try{
        $insercion = $this->pdo->prepare("update videojuego set titulo
            =:titulo, genero=:genero, precio=:precio where id=:id");
        $insercion->bindParam(':id', $i);
        $insercion->bindParam(':titulo', $t);
        $insercion->bindParam(':genero', $g);
        $insercion->bindParam(':precio', $p);
        $insercion->execute();
        return true;
    }catch (PDOException $e)
    {
        die($e->getMessage());
    }
}
```

2.4 Insertar (CREATE)

```
public function Insertar($i,$t,$g,$p)
{
    try{
        $insercion = $pdo->prepare("INSERT INTO videojuego(titulo,
            genero,precio) VALUES(:titulo, :genero, :precio)");

        $insercion->bindParam(':titulo', $t);
        $insercion->bindParam(':genero', $g);
        $insercion->bindParam(':precio', $p);
        $insercion->execute();
        return true;
    }catch (PDOException $e)
    {
        die($e->getMessage());
    }
}
```

3 El Controlador

Es el intermediario entre la vista y el modelo. Controla las interacciones del usuario en la vista. Pide los datos al modelo y los devuelve a la vista para que los muestre. Es el encargado de realizar las llamadas a las clases y los métodos. Crearemos los programas `insertar.php`, `editar.php`, `borrar.php`, `editar.php` con las estancias a los métodos específicos.

3.1 Insertar

```
<?php
//*****
//***** Insertar *****
//*****

include_once ("Videojuego.php");
$pdo = new Videojuego();

$consulta = $pdo->Insertar($_REQUEST['titulo'],$_REQUEST['genero'],
    $_REQUEST['pvp']);

if(!$consulta)
echo "<p class=\"aviso\">Error al ejecutar la consulta. SQLSTATE[{$pdo->
    errorCode()}]: {$pdo->errorInfo()}</p>\n";

$pdo = null;
header("Refresh:1; url=listar.php");
?>
```

3.2 Borrar

```
<?php
//*****
//***** Borrar *****
//*****

include_once ("Videojuego.php");
$pdo = new Videojuego();

$consulta = $pdo->Borrar($_REQUEST['titulo']);
if(!$consulta)
echo "<p class=\"aviso\">Error al ejecutar la consulta. SQLSTATE[{$pdo->
    errorCode()}]: {$pdo->errorInfo()}</p>\n";

$pdo = null;
header("Refresh:1; url=listar.php");
echo '<p>En breve le redirigiremos al listado.</p>';
?>
```

3.3 Editar

```
<?php
//*****
//***** Editar *****
//*****
```

```

include_once ("Videojuego.php");
$pdo = new Videojuego();

$consulta = $pdo->Editar($_REQUEST['id'],$_REQUEST['titulo'],$_REQUEST['genero'],$_REQUEST['precio']);

if(!$consulta)
echo "<p class='aviso'>Error al ejecutar la consulta. SQLSTATE[{$pdo->errorCode()}]: {$pdo->errorInfo()}</p>\n";

$pdo = null;
header("Refresh:1; url=listar.php");
echo '<p>En breve le redirigiremos al listado.</p>';
?>

```

3.4 Select

```

<?php
//*****
//***** Leer(Select) *****
//*****
include_once ("Videojuego.php");
$pdo = new Videojuego();

$consulta = $pdo->Listar();

echo "<a href='inicio.php'><img src='box-arrow-in-down.svg' width='32' height='32'></a>";

echo "<table class='table'><thead>";
echo "<tr> <th scope='col'>Nombre</th><th scope='col'>genero</th><th scope='col'>PVP</th><th scope='col'>operaciones</th></tr>";
echo "</thead><tbody>";
foreach($consulta as $registro){
    $titol=$registro['titulo'];
    ...
}
?>

```

Fijaos todos crean una estancia de videojuego `$pdo = new Videojuego();` y ya accedemos a las propiedades de la tabla videojuego.

4 La Vista

Muestra la información al usuario de manera lógica y legible.


```
//Formulario insertar
<?php
include_once("header.php");
?>

    <div class="row">
        <div class="col-sm-8"><h2>Agregar <b>Videojuego</b></h2></div>

        </div>
    </div>
    <div class="row">
        <form action="insertar.php" method="post">
            <div class="col-md-6">
                <label>Titulo:</label>
                <input type="text" name="titulo" id="titulo" class='form-control' maxlength="100" required >
            </div>
            <div class="col-md-6">
                <label>Genero:</label>
                <input type="text" name="genero" id="genero" class='form-control' maxlength="100" required>
            </div>
            <div class="col-md-3">
                <label>PVP:</label>
                <input type="real" name="pvp" id="pvp" class='form-control' required></textarea>
            </div>

            <div class="col-md-12 pull-right">
                <hr>
                <button type="submit" class="btn btn-success">Guardar datos</button>
            </div>
        </form>
    </div>
</div>
<footer>
    <div class="badge bg-primary text-wrap" style="width: 6rem;">
        This text should wrap.
    </div>
</footer>
</body>

//Formulario Listar
<?php
include_once("header.php");
?>

    <div class="row">
```

```
<div class="col-sm-8"><h2>Listado de VideoJuegos</h2>
</div>

</div>
</div>
<?php
include_once ("Videojuego.php");
$pdo = new Videojuego();

$consulta = $pdo->Listar();

echo "<a href='inicio.php'><img src='box-arrow-in-down.svg' width='32'
height='32'></a>";

echo "<table class='table'><thead>";
echo "<tr> <th scope='col'>Nombre</th><th scope='col'>genero</th><th
scope='col'>PVP</th><th scope='col'>operaciones</th></tr>";
echo "</thead><tbody>";
foreach($consulta as $registro){
    $titol=$registro['titulo'];

    echo "<tr><td>".$registro['titulo'].</td><td>".$registro['genero'].<td>".$registro['precio'].
    </td><td><a href='borrar.php?titulo=$titol'><img src='trash-sharp.svg'
    ' width='32' height='32'></a>".
    <a href=form_editar.php?id=".$registro['id'].><img src='create-sharp
    .svg' width='32' height='32'></a></td>".
    </tr>";
}
echo "</tbody></table>";

$pdo = null;
?>
```

5 Login & Password

Para manejar un sistema completo de login y password con contraseñas cifradas, necesitamos un método que cifre esos strings que el usuario introduce como contraseña; tanto en el formulario de registro como en el del login, ya que al codificar una contraseña, después tenemos que decodificarla para comprobar que ambas contraseñas (la que introduce el usuario en el login y la que tenemos en la base de datos) coincidan.

Necesitamos pues:

- `password_hash()` para almacenar la contraseña en la base de datos a la hora de hacer el INSERT
 - `PASSWORD_DEFAULT` almacenamos la contraseña usando el método de encriptación bcrypt
 - `PASSWORD_BCRYPT` almacenamos la contraseña usando el algoritmo CRYPT_BLOWFISH compatible con crypt()
- `password_verify()` para verificar el usuario y la contraseña

```
<?php
// Almacenando usuario y password en BD

$usu = $_POST["usuario"];
$pas = $_POST["password"];

$sql = "INSERT INTO usuarios(usuario, password) VALUES (:usuario, :
    password)";

$sentencia = $conexion -> prepare($sql);

$isOk = $sentencia -> execute([
    "usuario" => $usu,
    "password" => password_hash($pas,PASSWORD_DEFAULT)
]);
```

Ahora que tenemos el usuario codificado y guardado en la base de datos, vamos a recuperarlo para poder loguearlo correctamente.

```
<?php
// Recuperando usuario y password en BD

$usu = $_POST["login"] ?? "";

$sql = "select * from usuarios where usuario = ?";

$sentencia = $conexion -> prepare($sql);
$sentencia -> execute([$usu]);

$usuario = $sentencia -> fetch();

if($usuario && password_verify($_POST['pass'], $usuario['password']))
{
    echo"OK!";
} else {
    echo"KO";
}
```